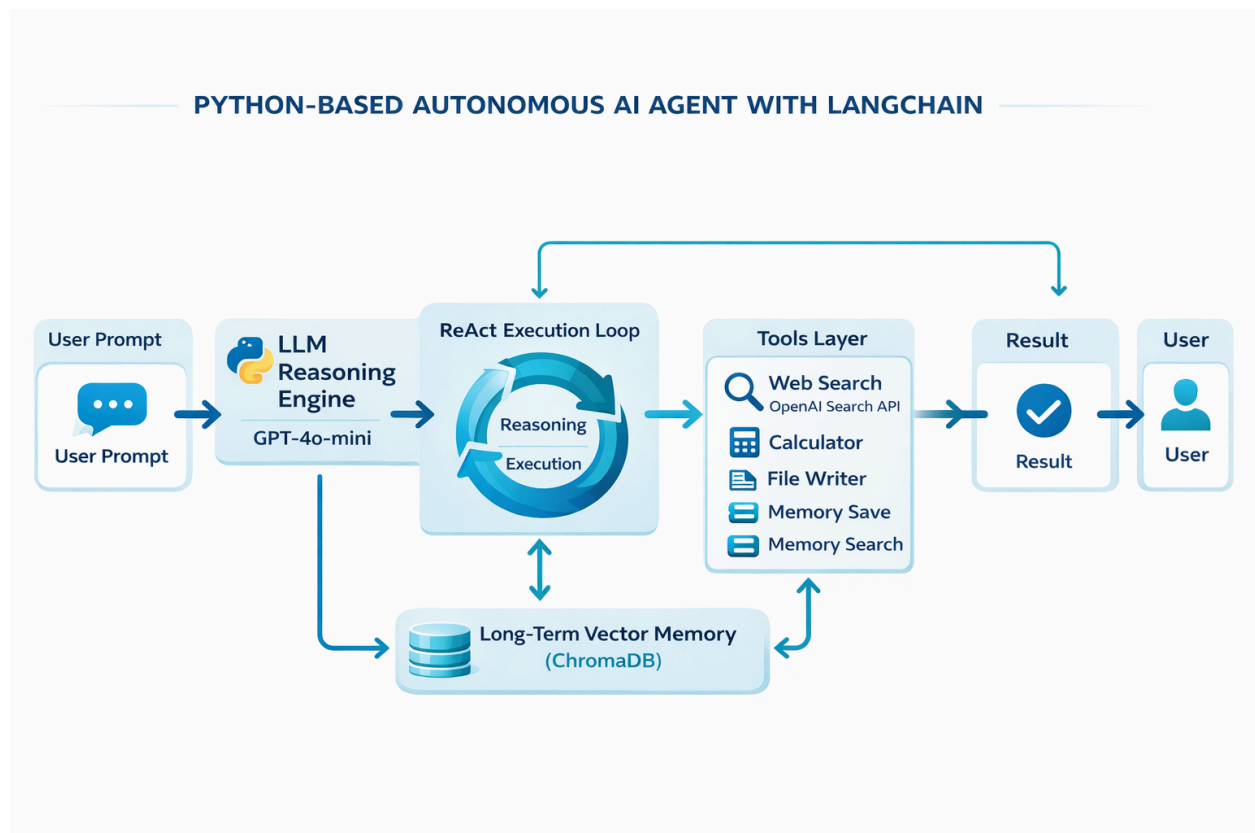


Build Your First Agent

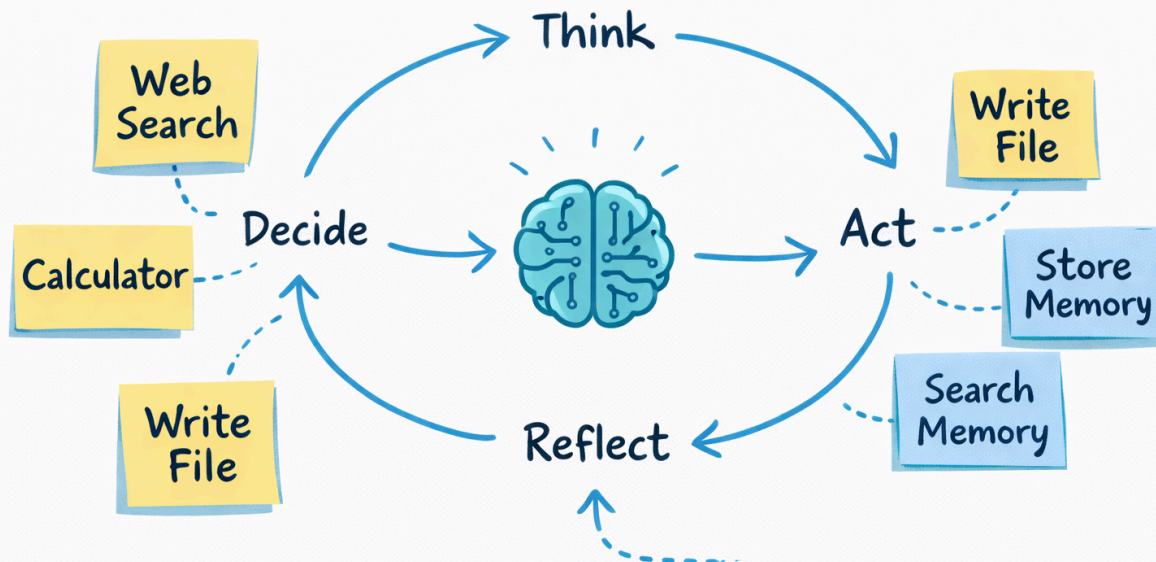
Goal: Build a modern autonomous AI agent using Python + LangChain v1, capable of reasoning, using tools, searching the web, storing long-term memory, and performing actions.

By the end, your agent will have:

- LLM reasoning
- Tool calling
- Web search (OpenAI Search API)
- Calculator
- File writer
- Long-term vector memory (Chroma)



ReAct Reasoning in an AI Agent



Prerequisites

1. Python

- Python 3.10 or later

2. OpenAI API key

Get an API key from OpenAI and set it:

macOS / Linux:

```
export OPENAI_API_KEY="sk-xxxxxxx"
```

Windows PowerShell:

```
setx OPENAI_API_KEY "sk-xxxxxxx"
```

```
$env:OPENAI_API_KEY="sk-xxxxxxx" (updated recently)
```

Then open a **new terminal** so the environment variable is picked up.

3. Create project folder + venv [for simplicity we skip for Non Python background student]

```
mkdir lab3-agent  
cd lab3-agent  
  
python3 -m venv venv  
source venv/bin/activate # Windows: venv\Scripts\activate
```

⚠ From now on, run all **pip** and **python** commands inside this activated venv.

4. Install required packages

```
pip install -U langchain langchain-openai langchain-community langchain-chroma openai
```

STEP 1 — Create **agent.py**

In the project root:

```
touch agent.py
```

Open it in your editor (VS Code, etc.).

STEP 2 — Basic LLM Setup

At the top of **agent.py**, add:

```
from langchain_openai import ChatOpenAI, OpenAIEmbeddings  
  
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    temperature=0.2,  
)
```

✅ This is your core reasoning engine.

(Optional sanity test later, but we'll do that once tools are added.)

STEP 3 — Add Memory Backend (ChromaDB)

Still in `agent.py`, below the LLM:

1. Import Chroma:

```
from langchain_chroma import Chroma
```

2. Create embeddings + vector store:

```
syst
```

This gives you a persistent vector database called `agent-memory`.

STEP 4 — Add Tool Imports

You will use:

- `@tool` decorator for tools
- OpenAI client for search

At the top (after other imports), add:

```
from langchain.tools import tool  
from openai import OpenAI
```

Create a single OpenAI client instance:

```
oai_client = OpenAI()
```

STEP 5 — Define Tools One by One

5.1 Web Search Tool (OpenAI Search API)

```
@tool("web_search", description="Search the web using OpenAI's built-in search tool.")
def web_search(query: str) -> str:
    """Use OpenAI Search to get up-to-date information."""
    response = oai_client.responses.create(
        model="gpt-4o-mini",
        input=query,
        tools=[{"type": "web_search"}],
        tool_choice={"type": "web_search"},
    )

    # Prefer structured output if available
    if hasattr(response, "output_text"):
        return response.output_text

    return str(response)
```

5.2 Calculator Tool

```
@tool("calculator", description="Evaluate a math expression.")
def calculator(expression: str) -> str:
    """Evaluate a mathematical expression and return the result as text."""
    try:
        return str(eval(expression))
    except Exception:
        return "Error evaluating expression."
```

⚠ This is just for lab use; in production you'd use a safe math parser, not `eval`.

5.3 File Writer Tool

```
@tool("file_writer", description="Write content into output.txt.")
def file_writer(content: str) -> str:
    """Write the given content into output.txt and return a status message."""
    with open("output.txt", "w") as f:
        f.write(content)
    return "File saved to output.txt"
```

5.4 Memory Tools — Save + Search

Use the `memory_db` from Step 3.

```
@tool("memory_save", description="Store information in long-term vector memory.")
def memory_save(text: str) -> str:
    """Store the given text in the agent's long-term memory."""
    memory_db.add_texts([text])
    return "Memory saved."

@tool("memory_search", description="Retrieve information from long-term memory.")
def memory_search(query: str) -> str:
    """Search long-term memory and return the top matching items."""
    results = memory_db.similarity_search(query, k=3)
    return "\n".join([r.page_content for r in results])
```

STEP 6 — Register Tools in a List

Below all tool definitions:

```
tools = [web_search, calculator, file_writer, memory_save, memory_search]
```

This is the list you'll pass into the agent.

STEP 7 — Create the Agent with `create_agent`

Import `create_agent`:

```
from langchain.agents import create_agent
```

Then create the agent:

```
agent = create_agent(
    model=llm,
    tools=tools,
)
```

STEP 8 — Add a Helper Function to Talk to the Agent

Add this near the bottom of `agent.py`:

```
def ask_agent(prompt: str) -> str:
    """Send a user message to the agent and return the final answer."""
    print(f"\n---- ASK AGENT ----\n{prompt}\n-----")
    state = agent.invoke({
        "messages": [{"role": "user", "content": prompt}]
    })
    print("---- RAW STATE ----")
    print(state)
    print("-----")
    return state["messages"][-1].content
```

STEP 9 — Add Test Calls (Main Script)

Finally, at the end of `agent.py`, add a simple script section:

```
if __name__ == "__main__":
    # Test 1: Web search + reasoning
    print(
        ask_agent("Find top 3 AWS security updates today and summarize them.")
    )

    # Test 2: Planning-only task
    print(
        ask_agent("Plan a 3-step workflow to generate a monthly financial report.")
    )

    # Test 4: Recall memory
    print(
        ask_agent("memory_search: What do you know about our monthly report?")
    )

    # Test 5: File writer
    ask_agent("file_writer: Write a summary of what this agent has done so far.")
```

Now the file is executable as a script.

STEP 10 — Run the Lab

In your terminal (venv still active):

```
python3 agent.py
```

Watch the console:

- You should see `----- ASK AGENT -----` logs
- Then raw `state` output for each call
- Then the agent's final text answers

```
This workflow can be adapted based on specific organizational needs and tools available.

----- ASK AGENT -----
memory_save: Remember that our monthly report is due every 25th.

----- RAW STATE -----
{'messages': [HumanMessage(content='memory_save: Remember that our monthly report is due every 25th.', additional_kwargs={}, response_metadata={}, id='0889fd7c-588c-4ca3-8888-2536e2a7a751', AIMessage(content='', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 23, 'prompt_tokens': 142, 'total_tokens': 165, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0}, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}), 'model_provider': 'openai', 'model_name': 'gpt-4o-mini-2024-07-18', 'system_fingerprint': 'fp_c4585b5b9c', 'id': 'chatcmpl-Cx5VcNYMAJa8ze1N1Q00iEdNzCxi', 'service_tier': 'default', 'finish_reason': 'tool_calls', 'logprobs': None, id='lc_run-019bb0d8-e7bb-7578-875e-97c08e828d3-0', tool_calls=[{'name': 'memory_save', 'args': {'text': 'Our monthly report is due every 25th.'}, 'id': 'call_ntnsQU6W395bwyOeyZK1386', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 142, 'output_tokens': 14, 'total_tokens': 156, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 0}}), ToolMessage(content='Memory saved.', name='memory_save', id='bc841814-cdad-4d79-91a5-edf8a4f18b8', tool_call_id='call_ntnsQU6W395bwyOeyZK1386'), AIMessage(content='I've noted that the monthly report is due every 25th.', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 14, 'prompt_tokens': 176, 'total_tokens': 190, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0}, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}), 'model_provider': 'openai', 'model_name': 'gpt-4o-mini-2024-07-18', 'system_fingerprint': 'fp_c4585b5b9c', 'id': 'chatcmpl-Cx5VdPrUD2LhsZs2SnRYC88wCNTq', 'service_tier': 'default', 'finish_reason': 'stop', 'logprobs': None, id='lc_run-019bb0d8-e7bb-7578-875e-97c08e828d3-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 176, 'output_tokens': 14, 'total_tokens': 190, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 0}})]}

----- ASK AGENT -----
memory_search: What do you know about our monthly report?

----- RAW STATE -----
{'messages': [HumanMessage(content='memory_search: What do you know about our monthly report?', additional_kwargs={}, response_metadata={}, id='6c6f7d68-3864-4873-8eac-ed96a77f2cae5'), AIMessage(content='', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 15, 'prompt_tokens': 139, 'total_tokens': 154, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0}, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}), 'model_provider': 'openai', 'model_name': 'gpt-4o-mini-2024-07-18', 'system_fingerprint': 'fp_c4585b5b9c', 'id': 'chatcmpl-Cx5Veumt68MbCSSLMywiaquIT9V', 'service_tier': 'default', 'finish_reason': 'tool_calls', 'logprobs': None, id='lc_run-019bb0d8-f247-7be1-adce-d8807b5de1a-0', tool_calls=[{'name': 'memory_search', 'args': {'query': 'monthly report'}, 'id': 'call_e7ABL72eStyDy37o3qJusIh', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 139, 'output_tokens': 15, 'total_tokens': 154, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 0}}), ToolMessage(content='Our monthly report is due every 25th.\nOur monthly report is due every 25th.', name='memory_search', id='1d91c542-344e-488e-a221-cbfc7774e378', tool_call_id='call_e7ABL72eStyDy37o3qJusIh'), AIMessage(content='The monthly report is due every 25th.', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 11, 'prompt_tokens': 182, 'total_tokens': 193, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0}, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}), 'model_provider': 'openai', 'model_name': 'gpt-4o-mini-2024-07-18', 'system_fingerprint': 'fp_c4585b5b9c', 'id': 'chatcmpl-Cx5Vgsez7hyDXap5Ttywa8cWig8n', 'service_tier': 'default', 'finish_reason': 'stop', 'logprobs': None, id='lc_run-019bb0d8-f7d9-72f3-8048-50e240b68926-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 182, 'output_tokens': 11, 'total_tokens': 193, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 0}})]}
```

Verify:

1. **Web results** appear for the AWS security question
2. The agent returns a **3-step workflow** for the report
3. Memory save doesn't error
4. Memory search returns something like "our monthly report is due every 25th"
5. `output.txt` is created in your project folder

Check the file:

```
cat output.txt
```

You should see a summary of what the agent has done.

```
(venv) ~ - lab1-agent open output.txt
(venv) ~ - lab1-agent cat output.txt
The agent has engaged in various tasks, including:

1. **Web Search**: The agent has the capability to perform web searches using OpenAI's built-in search capabilities to gather information on various topics.
2. **Math Evaluation**: The agent can evaluate mathematical expressions, providing accurate calculations based on user queries.
3. **File Writing**: The agent can write content into a text file, allowing for the storage of information or summaries as requested by the user.
4. **Memory Management**: The agent can store information in long-term memory for future reference and retrieve stored information when needed.

Overall, the agent has demonstrated its ability to assist with information retrieval, calculations, content creation, and memory management.
```

Reflection Questions

1. **Where does the agent show autonomy?**
2. **Why is long-term memory important?**
3. **How could you improve this agent?**
4. **What's needed for production?**

Conclusion

Your agent now has:

- LLM reasoning
- Web search (OpenAI Search API)
- Tool calling
- Persistent long-term memory
- File-writing capability
- ReAct-style execution